

Separable NM-ROM: architecture, training, and why linear problems need no quadrature

A study deck — what is learned, what is frozen, what is exact, what is sampled,
and what each of it costs per resolution

Tunable-NM-ROM project

29 August 2026; revised 30 August 2026 — added the PDE primer, the symbol reference,
the cross-PDE cost ladders, the Stokes rehearsal, and two framing corrections

Numbers: single-seed, f64, gate-checked runs (A100/H200 for Burgers and Poisson; CPU-timed NumPy paths for Stokes).
Burgers/Poisson tables reproduced from the generated presentation notes (28 Aug 2026); the cost ladders and Stokes tables
are `\input from tables/`, emitted from the run JSONs by `gen_2026-08-30-cross-pde-cost.py`.

The question this deck answers

The setup

A PDE solution $u(x)$ on a fine grid (n up to 2.6×10^5 points) is represented by a **small latent vector** $z \in \mathbb{R}^K$ ($K = 8$ or 16) through a **decoder**. The reduced-order model (ROM) solves the PDE *for* z , never for the grid values directly.

The catch

The PDE residual is a *sum over the whole grid*. If every solver iteration had to decode the field on the grid and sum it, the ROM would cost as much as the full solver.

What we will see

1. How the decoder is built so the grid can be *factored out*.
2. How it is trained (stage 1), then frozen.
3. **Linear PDE (Poisson)**: the whole residual collapses to a small precomputed table — *no sampling, no decoding, exact*.
4. **Nonlinear PDE (Burgers)**: one term refuses to collapse — we *sample* it at a few points and *learn where* (stage 2).
5. **The next rung (new, 29 Aug)**: a *quadratic* term collapses too — into a precomputed table — so Burgers becomes sample-free, and stage 2 disappears.

Primer 1/6 — Poisson: steady diffusion

How to read the next six slides: each class is scored by the *degree of its residual as a polynomial in the state*. Degree 1 and degree 2 precompute exactly; anything else must be sampled.

In plain language

A metal plate with a heater held against it. Wait until nothing changes any more. What you are left with is the Poisson solution: heat spreads until every point is the average of its neighbours, except where the heater puts heat in.

There is no time in it. You ask one question — “given this heater, what is the final temperature?” — and answer it with one solve.

The equation — *elliptic, linear*

$$-\Delta u(x) = f(x), \quad u = 0 \text{ on the boundary}$$

f is the heat source, u the temperature, Δ the Laplacian — “how far a point sits below its neighbourhood average”.

Residual degree **1** — so, for us

Every term is linear in u , so the whole projected residual is a fixed matrix times $h(z)$. **Zero sample points, exact to machine precision.** The easiest case there is.

Primer 2/6 — Heat: the same plate, watched over time

In plain language

The same plate — but now take the heater away and watch it cool, frame by frame. Sharp features smooth out quickly and never come back: diffusion *forgets*. A hot spot spreads into a broad bump within a few steps.

Because it forgets, a handful of smooth shapes describes almost every state it ever passes through. That is exactly the property a reduced model needs.

The equation — *parabolic, linear*

$$u_t = \alpha \Delta u$$

One implicit (backward-Euler) step is a Poisson-like solve: $(I - \Delta t \alpha \Delta) u^{n+1} = u^n$.

Residual degree 1 — so, for us

Poisson plus a time loop: still one precomputed matrix per operator, still zero sample points. **Honest caveat:** this project has no per- N cost ladder for heat, and on heat a *direct* reduced linear solve already beats the full solver — see the cost section.

Primer 3/6 — Wave: a drum head, and why it defeats us

In plain language

A stretched membrane, struck once. The bump splits, travels, reflects off the rim and comes back — for ever. Nothing is smoothed away, because **energy is conserved**.

Conservative is exactly the problem

- Diffusion forgets, so few shapes suffice. A wave forgets nothing: placing a travelling front anywhere needs *many* shapes — the Kolmogorov width decays slowly.
- Every ROM error is likewise conserved: it never decays, it accumulates.

The equation — *hyperbolic, linear, conservative*

$$u_{tt} = c^2 \Delta u$$

Residual degree 1 — so, for us

The residual machinery would work perfectly here: degree 1, quadrature-free, nothing to sample. **It is the decoder, not the residual, that fails.** Measured twice in this project (14 and 16 Aug): the rollout loses its energy and the verdict was “not fixable by tuning”. The cell was **retired without re-implementation** on 30 Aug.

Being easy for the residual does not make a PDE easy for a ROM.

Primer 4/6 — Burgers: the simplest nonlinear PDE

In plain language

Traffic on a one-way road where every car drives at a speed equal to the local density. Fast parts of the wave catch the slow parts, the profile steepens, and a near-discontinuity — a **shock** — forms. The viscosity ν smears that shock across a few grid cells instead of letting it become infinitely sharp. It is the standard proving ground: nonlinear enough to be interesting, small enough to have a trustworthy reference solution.

The equation — *nonlinear (quadratic), dissipative*

$$u_t + \underbrace{u u_x}_{\text{advection}} = \underbrace{\nu u_{xx}}_{\text{diffusion}}$$

The field multiplies its own slope. That product is the entire difficulty.

Residual degree 2 — so, for us

Quadratic still precomputes, into a 3-index table instead of a matrix. **Zero sample points.** The one non-polynomial residue is the upwind switch $|u|$ — and on non-negative data $|u| = u$, which makes even that exact.

Primer 5/6 — Stokes: honey in a box

In plain language

Very slow, very thick flow — honey, glacier ice, a swimming bacterium. So viscous that inertia is negligible: stop pushing and it stops at once, with no coasting.

Two novelties over everything above. The state is a **vector** — a horizontal and a vertical velocity at every point, not one number. And the fluid is **incompressible**: whatever flows into a cell must flow out of it.

The equations — *linear, vector-valued, constrained*

$$-\nu \Delta \mathbf{u} + \nabla p = \mathbf{f}, \quad \nabla \cdot \mathbf{u} = 0$$

The pressure p does not evolve: it is the **Lagrange multiplier** that enforces $\nabla \cdot \mathbf{u} = 0$. That makes this a *constrained* system, not an ordinary evolution.

Residual degree **1** — so, for us

Quadrature-free, exactly as Poisson is. The value of the cell is not speed but **machinery**: vector-valued decoding, a divergence-free trial space, and eliminating p so the ROM never solves for it — all of which Navier–Stokes needs.

Primer 6/6 — Navier–Stokes: the target

In plain language

Stokes with the inertia put back: real fluid, at speeds where it swirls. Water round a bridge pier, air over a wing, smoke in a room. The new term $(\mathbf{u} \cdot \nabla) \mathbf{u}$ is the fluid *carrying itself along* — and it is the Burgers nonlinearity in vector form.

Why it is the target

It is the PDE people actually want reduced, and the first problem in this deck that is simultaneously nonlinear, vector-valued and constrained.

The equations — *nonlinear (quadratic), vector, constrained*

$$\mathbf{u}_t + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\nabla p + \nu \Delta \mathbf{u}, \quad \nabla \cdot \mathbf{u} = 0$$

Residual degree 2 — so, for us

Nothing new is required. The linear half is Stokes (degree 1, built and gated); the convective half is Burgers-shaped (degree 2, the tensor, built in 1D and 2D). Navier–Stokes is those two machineries composed — **which is why the two cells that cannot each be written up as a positive result are still worth running.** Not run yet.

The symbols, once — a reference card

symbol	what it is	typical value here
N	mesh resolution — grid points along one axis	1D 128...65 536; 2D 64...1024; Stokes 32...256
n	grid unknowns — <i>all</i> the numbers the full solver carries; N in 1D, $\sim N^2$ in 2D	up to 2.6×10^5
K	latent size — how many numbers stand in for a whole field; what the ROM solves for	8 or 16 (sweeps to 32)
R	bank width — how many fixed spatial shapes the field is mixed from	32 (1D Burgers) to 96 (Poisson); 512 in the headline 2D bank
M	test modes — how many equations the solver is handed per step (weak-form rows)	32 or 64; $M \geq R$ where a direct solve is used
μ	parameters — what changes from one problem instance to the next	viscosity ν ; source centre, width, amplitude
G	the bank table — the R shapes evaluated on a point set once, $G_{ij} = bc(x_i) g_j(x_i)$	$G \in \mathbb{R}^{n \times R}$, frozen after stage 1
$h(z)$ $u = G h(z)$	the head — the R mixing amounts, from the K latent numbers the field — “a mix of R fixed shapes; only the amounts depend on z ”	a small MLP + linear skip, $\mathbb{R}^K \rightarrow \mathbb{R}^R$ the identity everything rests on
Φ	test modes — sine modes = eigenvectors of the discrete Laplacian, $L\Phi = -\Phi\Lambda$	$\Phi \in \mathbb{R}^{n \times M}$, fixed, never trained
$A = \Phi^\top L G$	precomputed operator — one matrix <i>per linear operator</i> L : the grid, folded away	$M \times R$; $= -\Lambda B$ with $B = \Phi^\top G$

The last two rows together: $\Phi^\top (L G h(z)) = A h(z)$ — the regrouping that makes a linear PDE cost nothing per iteration.

The separable decoder

$$u(x; z) = bc(x) \langle g(x), h(z) \rangle = bc(x) \sum_{j=1}^R g_j(x) h_j(z)$$

- $g(x) \in \mathbb{R}^R$ — the **bank**: R spatial *shapes* (Fourier features $\rightarrow$ small MLP). Depends on x only.
- $h(z) \in \mathbb{R}^R$ — the **head**: MLP + linear skip from the latent to R *mixing amounts*. Depends on z only.
- $bc(x)$ — a fixed mask that makes the boundary condition hold exactly.
- $R = 32$ (Burgers), $R = 64\text{--}96$ (Poisson).

Why “separable” matters

The two tracks never see each other's input.

Evaluate the bank on any point set once and store it as a table

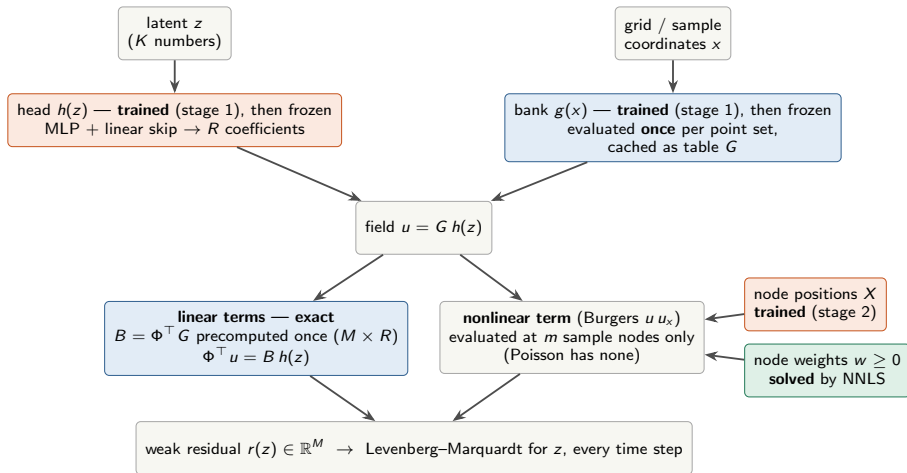
$$G \in \mathbb{R}^{n \times R}, \quad G_{ij} = bc(x_i) g_j(x_i).$$

Then the field on that point set is just

$$u(z) = G h(z) \quad (\text{table} \times \text{coefficient vector}).$$

The field is a mix of R fixed shapes; only the R mixing amounts depend on z . Everything downstream exploits exactly this.

Architecture diagram — who is trained, who is frozen, who is solved



■ trained by gradient ■ frozen / precomputed (exact path) ■ solved by a convex fit, never gradient-trained

Stage 1 — training the decoder (auto-decoder)

$$L_1 = \underbrace{\frac{\text{mean}_s \|G h(z_s) - u_s\|^2}{\text{mean} \|u\|^2}}_{\text{relative reconstruction}} + \lambda_{\text{orth}} \underbrace{\left\| \frac{G^\top G}{n s^2} - I \right\|_F^2}_{\text{keep the bank well-conditioned}}$$

Jointly optimise the bank g , the head h , and **one latent code** z_s **per training snapshot** u_s (there is no encoder — the codes are free variables). $\lambda_{\text{orth}} = 10^{-4}$; Adam, full batch, warm-up then cosine decay ($10^{-3} \rightarrow 10^{-5}$), 40 000 steps.

In words

- First term: “every training snapshot must be reconstructible from *some* 8-number code.”
- Second term: “the R shapes should be roughly orthogonal” — a conditioning nudge so the mixing amounts stay well-scaled; it does not change what the decoder can represent.

After stage 1

Bank, head, and codes are **frozen**. Nothing downstream ever changes the decoder. The training codes are discarded; the solver finds its own z at run time.

How the decoder is trained in each case — one recipe, three data sets

	Poisson 2D (linear)	Burgers 1D, sampled	Burgers 1D, tensor
What a snapshot is	the solution u of $-\Delta u = f$ for one source f (a Gaussian blob: centre $\sim U(0.15, 0.85)^2$, width log-uniform $[0.02, 0.1]$, amplitude $U(0.5, 2)$)	one time state of one trajectory: 512 trajectories (blob IC, $\nu \sim \log\text{-}U[0.01, 0.1]$) $\times$ 50 steps; 8192 states drawn	<i>identical to the sampled case</i>
Truth solver	conjugate gradients to 10^{-13}	tridiagonal Newton, backward Euler, residual $\leq 10^{-8}$	same committed checkpoints
Training snapshots	512 sources $\rightarrow$ 512 fields	8192 states	<i>nothing retrained</i>
Sizes $K / R / M$	16 / 64–96 / 64	8 / 32 / 32	8 / 32 / 32
Loss	L_1 (relative recon + λ_{orth} , 10^{-4})	same L_1	same L_1
Optimiser	Adam, warm-up then cosine, 60k–100k steps, seed 0	Adam, warm-up then cosine $10^{-3} \rightarrow 10^{-5}$, 40k steps, seed 0	same; new N trained in 283 s (16k) / 1012 s (65k)
Precision	f64, matmul precision highest	f64, highest	f64, highest (f32 would not do)
Then frozen	yes	yes	yes
Held out for grading	16 same-seed held-out sources + 16 fresh-seed sources	8 fresh trajectories (seed 1; parameter tuples ≥ 0.06 from any training one)	the same 8

Stage 1 never sees the PDE operator, the test modes, the quadrature, or the tensor: it only learns to *reconstruct snapshots* from a free code per snapshot. Everything PDE-specific happens after freezing — next slide.

How the decoder is used online — it defines a manifold, it does not predict

Weak form. Pick M test functions $\Phi \in \mathbb{R}^{n \times M}$ (sine modes = exact eigenvectors of the discrete Laplacian, $\Delta \Phi = \Phi \Lambda$). The residual handed to the solver is the PDE residual projected onto them:

$$r(z) = \Phi^\top \mathcal{R}(G h(z)) \in \mathbb{R}^M, \quad M = 32 \text{ or } 64.$$

Solve. Each time step: damped Levenberg–Marquardt over the K latent unknowns, driving $r(z) \rightarrow 0$; needs r and $\partial r / \partial z$.

The cost question

$\Phi^\top \mathcal{R}(u)$ is a **sum over all n grid points**. Done naively, each of thousands of solver iterations would

1. decode $u = G h(z)$ on the grid ($n \times R$ flops),
2. apply the PDE operator on the grid,
3. sum against M test modes.

That is the full-order cost, every iteration.

Two ways out

Exact: regroup the arithmetic so the grid disappears (works when the term is linear).

Approximate: estimate the sum from $m \ll n$ sample points with weights — *quadrature* (needed when the term is nonlinear).

Why a linear term needs no sampling

The one fact

A weighted sum of a mix equals the mix of the weighted sums.

$$\Phi^\top u = \Phi^\top \left(\sum_j h_j G_{:,j} \right) = \sum_j h_j (\Phi^\top G_{:,j}) = \underbrace{(\Phi^\top G)}_{B: M \times R} h(z)$$

The field is a mix of R fixed shapes, $u = \sum_j h_j(z) G_{:,j}$, and a linear residual term is a fixed weighted sum of the field — hence the line above. B does not depend on $z \Rightarrow$ compute it **once**, offline, on the full grid.

Online: an $M \times R$ matrix times R numbers.

Exact, not approximate. Nothing was estimated; it is the same arithmetic regrouped: $\Phi^\top (G h) = (\Phi^\top G) h$.

Shopping analogy

Every cart is a mix of the same 32 products in different amounts. You do not re-price the cart item by item each time; you look up 32 unit prices once, and the total is amounts $\times$ unit prices. B is the unit-price sheet: for each of M test modes and each of R shapes, “what this shape contributes to this mode.” $64 \times 32 = 2048$ numbers, computed once.

Poisson: the whole residual is a small table

$$\boxed{r(z) = W \odot [\Lambda B h(z) - f_M]} \quad \frac{\partial r}{\partial z} = W \odot \Lambda B \frac{\partial h}{\partial z}, \quad B = \Phi^\top G, \quad f_M = \Phi^\top f$$

Poisson: $-\Delta u = f$. Every term is linear in u .

- **Laplacian moved onto the test modes.** Because $\Delta \Phi = \Phi \Lambda$, $\Phi^\top (\Delta u) = (\Delta \Phi)^\top u = \Lambda \Phi^\top u$. The operator becomes M known numbers Λ — it never touches the field.
- **Source term projected once:** $f_M = \Phi^\top f$.
- $\partial h / \partial z$: autodiff of an $8 \rightarrow 32$ MLP.
- W : fixed per-mode scaling that balances the rows.

What runs online

- the tiny head $h(z)$,
- a 64×32 matvec,
- the head's 32×8 Jacobian.

No grid. No decode. No sample points. No fitted weights. The field $G h(z)$ is formed once, at the very end, to write out the answer.

What “0 sample points” means

The sampled (NNLS) path estimates the same M numbers from 256 chosen points — and gets them 5–8 % wrong at solutions. The quadrature-free path computes them exactly and does not need the points at all.

Poisson 2D results ($N = 128/256/512$, same solver, same frozen decoders)

N	path	sample pts	solve ms	solution err	resid. err vs full	grad. cos (min)	setup s
128	full grid	15 876	3.98	3.06e-2	0 (ref)	1.000	—
128	sampled (NNLS)	256	3.14	3.06e-2	7.9e-2	−0.61	28.6
128	quadrature-free	0	2.93	3.06e-2	4.2e-14	1.000	0.9
256	full grid	64 516	4.96	3.14e-2	0	1.000	—
256	sampled (NNLS)	256	2.75	3.14e-2	6.2e-2	−0.60	37.3
256	quadrature-free	0	2.68	3.14e-2	3.6e-14	1.000	0.7
512	full grid	260 100	11.11	3.15e-2	0	1.000	—
512	sampled (NNLS)	256	3.09	3.15e-2	5.5e-2	−0.65	25.4
512	quadrature-free	0	2.87	3.15e-2	3.3e-14	1.000	0.8

Exact: residual matches the full grid to ~ 13 digits at every solver solution (zero gate failures).

Fastest and flat in N : full grid grows 4 $\rightarrow$ 11 ms; quadrature-free stays ~ 2.9 ms.

Nothing to fit: the 25–37 s NNLS quadrature fit disappears; the sampled rule’s gradient could even point the wrong way ($\cos < 0$).

Two corrections to the framing — read these with the Poisson result

1. Non-symmetry is *not* what kills the diagonal shortcut

Poisson gets $\Phi^\top(\Delta u) = \Lambda \Phi^\top u$ because the columns of Φ are eigenvectors of Δ . It is tempting to conclude that a *non-symmetric* operator can have no such shortcut. **That is wrong.**

A non-symmetric L still gives $\Phi^\top L u = \Lambda \Phi^\top u$ whenever Φ holds its **left** eigenvectors, $L^\top \Phi = \Phi \Lambda$. Self-adjointness is *sufficient*, never necessary.

What actually kills the shortcut is simply Φ **not being eigenvectors of the operator in question** — and losing it costs nothing. The general form $A = \Phi^\top L G$, one $M \times R$ matrix per affine operator component, is still exact and still independent of z . Only the diagonal, an optimisation, goes.

2. On a linear PDE the nonlinear head can never buy *accuracy*

If the PDE is linear and its parameter dependence is affine, $L(\mu) = \sum_q \theta_q(\mu) L_q$, then the set of all its solutions is **literally a linear subspace** — spanned by the solves of finitely many right-hand sides.

A nonlinear decoder therefore cannot represent anything a linear basis of the same width cannot. **Its entire value on such a problem is cost.**

This is the correct reading of the Poisson quadrature-free result: the solution error is *identical* down all three rows of each block above — the decoder floor — and only the milliseconds and the NNLS fit moved. It also predicts the Stokes verdict later in this deck. The nonlinear head earns its keep only on a **nonlinear** PDE, or under **non-affine** parameter dependence.

Why a nonlinear term breaks the trick

Burgers: $u_t + u u_x = \nu u_{xx}$. The advection term multiplies the field by itself:

$$\Phi^\top (u \odot u_x) = \Phi^\top \left[\left(\sum_j h_j G_{:,j} \right) \odot \left(\sum_k h_k G'_{:,k} \right) \right] = \sum_{j,k} h_j h_k \Phi^\top (G_{:,j} \odot G'_{:,k})$$

- Mixing does not pass through a product: every *pair* of shapes appears (R^2 terms, $M \times R \times R$ table).
- Worse: our discretisation is **sign-upwind** — the stencil for u_x switches with the sign of u at each point. That is not a polynomial in h ; no table can hold it.

Consequence

The only way to know the term's value is to **actually look at the field somewhere**. So we must decode at *some* points — and then we have to choose *which* points and *what weights*: that is quadrature, and its error enters the ROM's error.

But only that one term

Time derivative and diffusion are linear $\Rightarrow$ they still go through $B = \Phi^\top G$, exactly (“exlin”). Sampling touches the advection term and nothing else.

What we do for Burgers: exact linear terms + sampled advection

Backward-Euler step from z^n to z ; $N(u) = u u_x$ (sign-upwind); m nodes X with weights $w \geq 0$; $\Phi_X, G_X =$ test modes and bank evaluated at the nodes:

$$r(z) = W \odot \left[\underbrace{B(h(z) - h(z^n))}_{\text{time derivative, exact}} + \Delta t \left(\underbrace{\Phi_X^\top (w \odot N_X(G_X h(z)))}_{\text{advection, sampled at } m \text{ nodes}} + \underbrace{\nu \wedge B h(z)}_{\text{diffusion, exact}} \right) \right]$$

The per-iteration cost

Decode at m nodes only ($m \times R$), one upwind stencil on m values, an $M \times m$ matvec, and the exact linear part. **Independent of the grid size n .** Measured: latent solve flat from $N = 128$ to 4096.

Choosing the nodes: baseline = NNLS

Given positions X , weights are fit by non-negative least squares so the sampled term reproduces the exact full-grid term on training states. Convex, no gradients, always available as a fallback. Positions themselves are the free choice — stage 2 learns them.

Stage 2 — learning where to sample (decoder frozen)

Only the node positions X are trainable. The loss is a **mismatch against the exact full-grid term**, in value *and* in the z -gradient the solver follows:

$$L_{\text{samp}} = \text{mean}_s \frac{\|W_s(\Phi_X^\top(w \odot N_X(u_s)) - \Phi^\top N(u_s))\|^2}{\|W_s \Phi^\top N(u_s)\|^2}, \quad L_{\text{jac}} = \text{same expression for } \partial_z(\cdot)$$

$$L_2 = \frac{L_{\text{samp}}}{L_{\text{samp}}^0} + \frac{L_{\text{jac}}}{L_{\text{jac}}^0} + w_{\text{sep}} \sum_{i < j} \text{relu}(d_{\text{min}} - |x_i - x_j|)^2$$

Optimisation

- Adam on X only, LR 3×10^{-3} , 2 000 steps.
- A sigmoid box keeps every node inside the domain.
- The last term keeps nodes apart (minimum separation).
- Each term $\div$ its starting value: both begin at 1, equal weight.

Weights are never gradient-trained

Every 500 steps the weights $w \geq 0$ are re-solved by NNLS on the same loss rows (variable projection). The convex NNLS rule is therefore always the fallback: learning can only move points, never damage the decoder.

Stage 2 — the design choices, and why

Teacher frozen

The full-grid advection term and the decoder cannot move, so the loss cannot be gamed by “fooling the points” — only better placement lowers it. (An earlier co-training variant that let the decoder move was negative.)

Match the gradient, not just the value

The solver steps along $\partial r / \partial z$. A rule that matches residual values but not their z -sensitivity can send the solver the wrong way — Poisson's NNLS rule had gradient cosines below zero.

Held-out states

A disjoint set of training states sits in no gradient step and no NNLS row; it monitors whether the placement generalises rather than memorises.

What the certification is

Rollouts on 8 fresh test trajectories the decoder never saw, graded against the full-order truth, compared to the same-budget NNLS rule and to the full-grid oracle.

1D Burgers results ($N = 128 \dots 4096$, 8 fresh trajectories, 50 implicit steps)

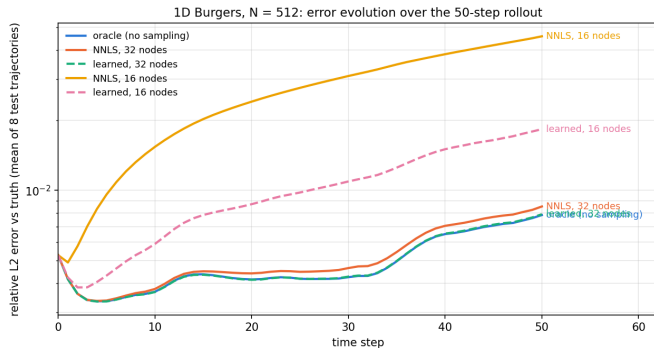
N	decoder floor	oracle	NNLS 32	learned 32 (vs NNLS)	NNLS 16	learned 16 (vs NNLS)
128	3.71e-3	5.01e-3	5.48e-3	5.02e-3 (−8.4%)	2.10e-2	1.01e-2 (−52%)
256	3.73e-3	6.17e-3	6.24e-3	6.20e-3 (−0.6%)	2.56e-2	1.07e-2 (−58%)
512	3.80e-3	4.88e-3	5.19e-3	4.89e-3 (−5.9%)	2.68e-2	1.03e-2 (−62%)
1024	3.79e-3	5.54e-3	5.62e-3	5.45e-3 (−2.9%)	2.22e-2	8.61e-3 (−61%)
2048	3.83e-3	5.08e-3	5.53e-3	5.15e-3 (−7.0%)	2.25e-2	8.44e-3 (−63%)
4096	3.83e-3	4.49e-3	4.89e-3	4.53e-3 (−7.3%)	2.29e-2	9.22e-3 (−60%)

Oracle = advection summed on the full grid (no sampling): the best any node set can do with this decoder. Flat across $32\times$ in resolution.

32 nodes: sampling costs 2–10 % over the oracle; learned nodes close 84–97 % of that gap — they land on the oracle.

16 nodes (starved): NNLS is 4–5 $\times$ the oracle error; learned placement cuts it by 52–63 %, six for six. It improves a budget; it does not replace a bigger one.

Error over the rollout ($N = 512$)



- All arms share the same initial-condition fit (step 0).
- Oracle and learned-32 are indistinguishable step for step.
- The starved NNLS rule (16 nodes) drifts from step 1 onward ($4.6\text{e-}2$ by step 50); learned nodes at the same budget hold it to $1.8\text{e-}2$.

ROM online cost per 50-step trajectory (A100), independent of resolution: 49–62 ms; 22–24 ms after inner-loop optimisation with errors unchanged to 10^{-9} .

Extending the linear trick one degree up

Recall why linear was free

$\Phi^\top (G h) = (\Phi^\top G) h = B h$ — a weighted sum of a mix is the mix of the weighted sums.

A **quadratic** term is a weighted sum of a product of two mixes. Expand both mixes and you get every *pair* of shapes — but there are only $R^2 = 1024$ pairs, and none of them depend on z :

$$\Phi^\top (u \odot D^- u) = \sum_{j,k} h_j h_k \underbrace{\sum_x \Phi_{xi} G_{xj} (D^- G)_{xk}}_{T_{ijk}} = h^\top T h, \quad T \in \mathbb{R}^{M \times R \times R} = 32 \times 32 \times 32.$$

Same shopping analogy, one level up

The cart total now depends on *pairs* of products (“buy j and k together”). There are 32×32 pairs; price every pair once, and any cart is a 32×32 lookup times the amounts. Still no need to walk the aisles.

Standard precomputed quadratic operator of POD–Galerkin / operator inference; new here only in the setting (nonlinear MLP head over a learned bank). Literature name: *exact projection of a polynomial nonlinearity, no hyper-reduction needed*.

What runs online

One contraction $h^\top T h$ ($\sim 30k$ flops) and its Jacobian $(Qh) \partial h / \partial z$, $Q = T + T^{(j \leftrightarrow k)}$ (T is not symmetric: bank vs. differenced bank). No grid, no sample points, no fitted weights, nothing trained. T is built **once**, offline — 304 ms even at $N = 65\,536$.

The obstacle was the stencil, and positivity removes it

The FOM's advection is **sign-upwind**: it looks left where $u > 0$ and right where $u < 0$. A switch is not a polynomial — no table can hold it. But the switch decomposes exactly:

$$N_{\text{upwind}}(u) = u D^c u - \frac{\Delta x}{2} |u| \Delta_h u.$$

Central difference plus a small artificial viscosity: the only non-polynomial piece is $|u|$.

On non-negative fields, $|u| = u$

Our 1D family is positive Gaussian blobs with zero walls; viscous Burgers keeps such data ≥ 0 (max principle), and so does the FOM. So on the *truth*, the backward-difference table T is the full-grid oracle, exactly. Gate T0: on decoded states that are entirely positive, agreement $\leq 8 \times 10^{-15}$.

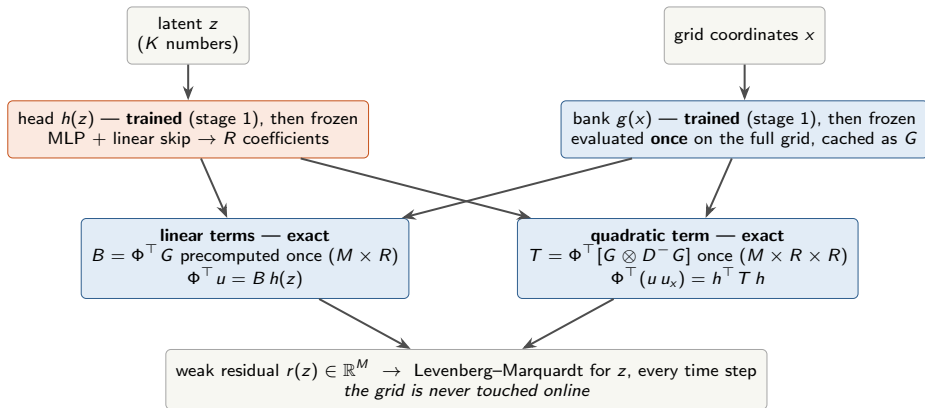
The only deviation: decoded undershoots

The ROM's decoded field dips slightly below zero at $\sim 7\%$ of points (min ≈ -0.02). There the FOM would flip the stencil and the table does not. The exact discrepancy is

$$q_T - q_{\text{or}} = -\Delta x \Phi^\top [\mathbb{1}[u \leq 0] u \Delta_h u],$$

first order in the undershoot, times Δx , times the blob's curvature — small because Δx is small. Bounded; measured at $0.4 \times$ the bound.

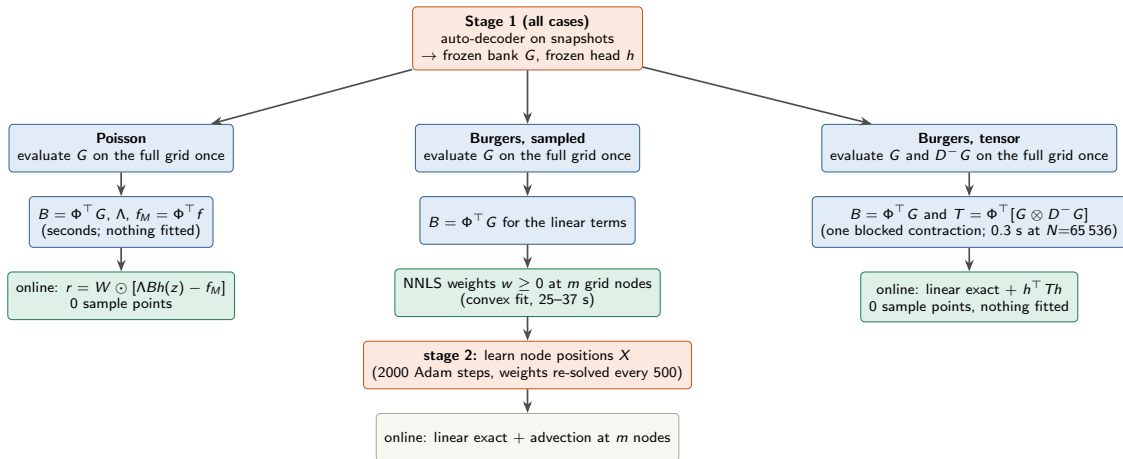
Architecture with the tensor — what changes, what does not



Training: nothing new. Stage 1 is unchanged (same loss L_1 , same 40 000 steps, same checkpoints — the results below reuse the *committed* decoders, nothing retrained).

Stage 2 disappears. No node positions to learn, no NNLS weights to fit, no budget m to choose, no held-out monitoring. The orange “nodes” box and the green “weights” box of the earlier diagram are gone; the blue path is the whole residual, as for Poisson.

After stage 1: what each case builds from the frozen decoder



■ gradient training ■ one-time exact precomputation from the frozen bank ■ convex fit / direct evaluation, no gradients

The decoder training is the same in all three columns; the columns differ only in what is *derived* from the frozen bank. Linear and quadratic terms are derived exactly; only a term that is neither needs the fitted (green) and learned (orange) steps of the middle column.

Results: the tensor lands on the oracle ($N = 128 \dots 1024$, committed checkpoints)

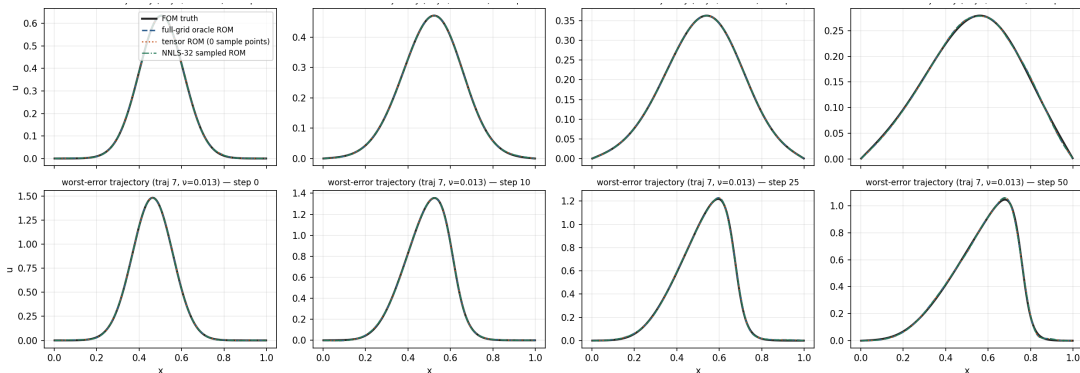
N	oracle err	tensor err	diff max/traj	NNLS-32 err	learned-32 err	e2e oracle	e2e tensor	e2e NNLS-32
128	5.009e-3	5.009e-3	1.2e-6	5.480e-3	5.021e-3	29.0 ms	24.4 ms	25.0 ms
256	6.165e-3	6.165e-3	6.4e-7	6.236e-3	6.197e-3	35.0 ms	29.2 ms	29.5 ms
512	4.877e-3	4.877e-3	2.5e-7	5.194e-3	4.886e-3	30.7 ms	27.5 ms	26.1 ms
1024	5.537e-3	5.537e-3	2.3e-7	5.615e-3	5.451e-3	27.4 ms	21.8 ms	22.0 ms

Accuracy. Equal to the full-grid oracle per trajectory to $\leq 1.2 \times 10^{-6}$ (direct field difference $\leq 9 \times 10^{-6}$); the solver takes the same number of Jacobian evaluations on 31 of 32 trajectories. Zero sample points.

Versus sampling. The tensor inherits the oracle's small margin over NNLS-32 — *not* distinguishable from zero at 8 trajectories (sign test $p \geq 0.07$). Equal to learned-32. The gain is exactness and simplicity, not accuracy at this budget.

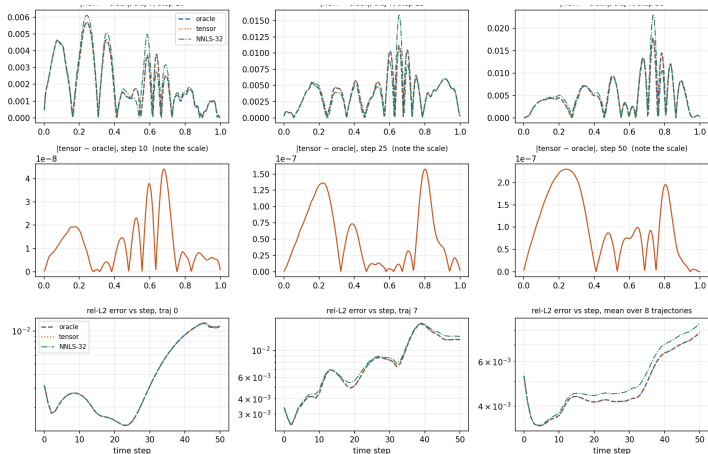
Cost. Within $\sim 7\%$ of the sampled rule end-to-end; 10–20 % under the oracle. No 1D speedup — everything here is launch-bound (see next slide). f64 required: in f32 the table's round-off would exceed the undershoot term.

Seeing it (1/2): 1D fields, $N = 512$ — truth and the three ROMs overlap



FOM truth (black) with the full-grid oracle (blue dashed), the tensor ROM (orange dotted, 0 sample points) and NNLS-32 (green dash-dot) at steps 0/10/25/50; top row the median-error trajectory ($\nu = 0.092$), bottom the worst ($\nu = 0.013$, steepest front). All four curves coincide at this scale: the ROMs sit on the decoder floor. Local reproduction of the A100 job, same numbers.

Seeing it (2/2): where the arms differ



Worst trajectory, $N = 512$. Top: $|\text{ROM} - \text{truth}|$ — tensor and oracle are one curve; NNLS-32 spikes higher at the front. Middle: $|\text{tensor} - \text{oracle}|$ on its own scale, 10^{-8} – 10^{-7} . Bottom: relative error per time step (two trajectories and the 8-trajectory mean) — tensor $\equiv$ oracle; NNLS-32 drifts above after step ~ 10 .

Results: the latent solve does not grow with N

One A100, one process, N alternating, reps outermost — latent solve,
ms per 50-step trajectory

N	oracle (full grid)	NNLS-32	tensor
128	21.4	16.5	17.1
256	26.2	19.9	20.2
512	21.7	14.8	15.7
1024	21.3	15.5	16.0
2048	20.4	15.9	14.4
4096	20.8	13.6	14.3
exponent	-0.04	-0.07	-0.08

Beyond the launch floor (separate jobs, same GPU model; decoders
trained in-job at the same 3.76×10^{-3} floor)

N	oracle	NNLS-32	tensor	tensor = oracle to
16 384	21.4	13.9	14.2	5.8×10^{-10}
65 536	32.9	13.5	13.8	1.7×10^{-10}

Why the oracle looks flat at first

Each solver iteration is ~ 15 tiny GPU operations, each with a $5\text{--}10\ \mu\text{s}$ launch overhead. Decoding 4096 points takes $< 1\ \mu\text{s}$ of actual work — invisible. Below $\sim 10^4$ points wall time counts *operations*, not bytes: the “launch floor”.

Why the tensor stays flat forever

Its per-iteration work is a 32^3 table times a 32-vector. N is not in its shape. Only the offline build sees the grid.

Honest scope

“Constant” = the *latent solve*. The IC projection and the final decode still touch the grid once per trajectory ($6\text{--}9\ \text{ms} + < 1\ \text{ms}$ here, flat only because launch-bound). The $16\text{k} \rightarrow 65\text{k}$ numbers are a cross-job ratio, not a fitted exponent.

2D results: tensor = oracle, and the full-grid cost is now visible

2D Burgers, $K=16$, $R=64$, $M=64$; tensor = 64^3 (2 MB) covering both axes; truth asserted ≥ 0 at every N

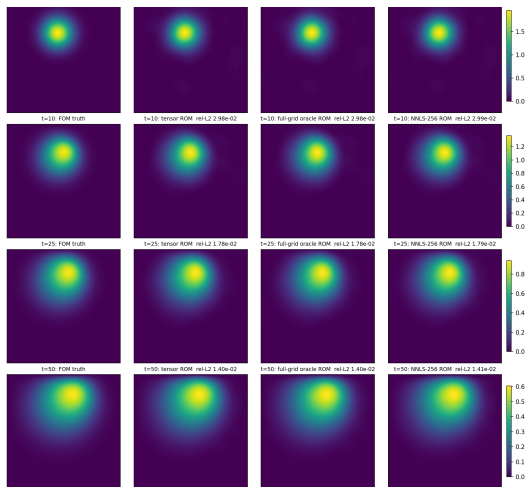
N	GPU	oracle err	tensor err	NNLS-256 err	diff max	oracle solve	NNLS-256 solve	tensor solve
64	A100-40	2.309e-2	2.309e-2	2.319e-2	8e-5	32.6 ms	28.8	27.5
256	A100-80	2.451e-2	2.451e-2	2.450e-2	7e-6	74.1	29.1	27.7
512	H200	2.456e-2	2.456e-2	2.450e-2	2e-6	97.3	20.5	20.0
1024	A100-80	6.381e-2	6.381e-2	6.391e-2	2e-5	670.0	29.4	27.1
paired vs matched-accuracy FOM				$N=64$: $0.42\times$	256: $0.58\times$	512: $0.98\times$	1024: $3.45\times$ (sampled rule $3.31\times$)	

Accuracy. Tensor = oracle at every N , with identical stop reasons *and* identical LM attempt counts. Not better than NNLS-256 (ratio 0.996–1.003): at $m = 4M$ sampling was never binding, so the tensor matches it with nothing fitted (build 2 s vs an 11–26 s NNLS fit).

Cost. The $O(N)$ oracle climbs $33 \rightarrow 670$ ms over $16\times$ in resolution; the tensor stays at 20–28 ms, 2–8 % under the sampled rule. Different GPU per N — within-job ratios, not an exponent. The 2D crossover vs the FOM is unchanged: win at 1024^2 , loss below.

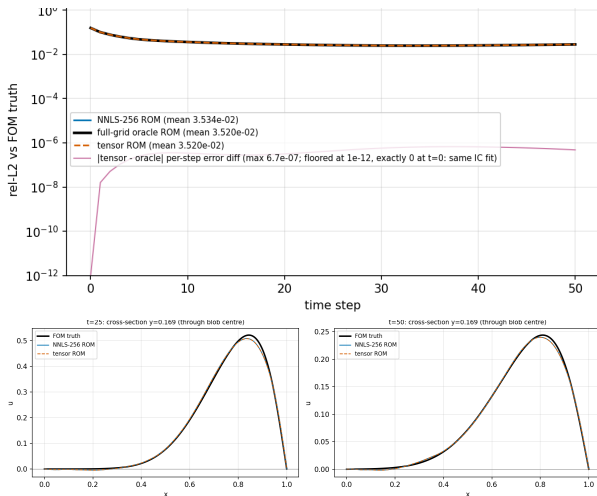
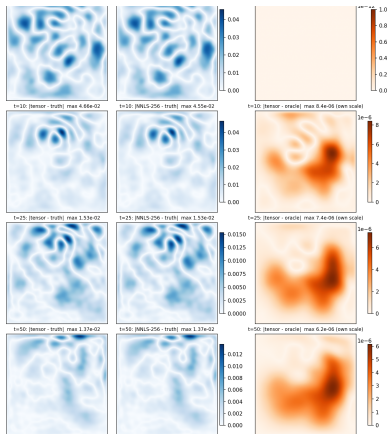
Limits. Decoded undershoots are larger in 2D (15–17 % of points), so parity is 10^{-4} – 10^{-6} . The $R=512$ headline bank does *not* compress by head-PCA (499/512 directions needed; tripwire stopped it) — that configuration stays sampled. The $N=1024$ checkpoint is a weak inherited decoder (its 6.4×10^{-2} is the checkpoint, not the residual).

Seeing it in 2D (1/2): fields, $N = 256$ — truth and the three ROMs



Test trajectory 5 ($\nu = 0.034$): FOM truth | tensor ROM | full-grid oracle ROM | NNLS-256 ROM at steps 0/10/25/50, shared colour scale per row. Mean rel-L2: tensor 2.141×10^{-2} , oracle 2.141×10^{-2} , NNLS-256 2.151×10^{-2} . The four panels are indistinguishable.

Seeing it in 2D (2/2): where the arms differ



Left ($N = 256$): $|\text{tensor} - \text{truth}|$ and $|\text{NNLS-256} - \text{truth}|$ on a shared 10^{-2} scale are the same picture; $|\text{tensor} - \text{oracle}|$ on its own scale is 10^{-6} (0 at $t = 0$: same IC fit). Right ($N = 1024$): error per step is one curve for all arms, tensor-oracle gap $\leq 7 \times 10^{-7}$; cross-sections at $t = 25, 50$ — the ROMs coincide and under-resolve the peak slightly (decoder floor, not residual).

Stokes 2D — the machinery is exact, and the verdict is negative on purpose

Exactness — gates S4 (against an *independent* strong-form full-grid path) and the S7 pressure rung

N	residual	Jacobian	pressure elim. vs FOM p	rank A	cond A
32	6.0e-16	4.7e-16	7.1e-17	32	26.7
64	1.8e-15	1.1e-15	1.3e-16	32	36.3
128	2.3e-15	1.6e-15	2.8e-16	32	45.2
256	7.4e-15	4.8e-15	5.4e-16	32	50.9

Accuracy — 64 held-out parameters, one mass-weighted centred norm, $K = 8$, $R = 32$

N	nonlinear QF ROM	POD-8	vs POD-8	direct G -span	POD-32 Galerkin	QF / G -span err	QF / G -span time
32	1.890e-02	5.013e-02	2.65×	2.91e-14	2.78e-14	6.5e+11	5387×
64	1.893e-02	4.981e-02	2.63×	4.56e-14	4.18e-14	4.1e+11	7038×
128	1.866e-02	4.833e-02	2.59×	3.34e-13	2.85e-13	5.6e+10	6743×
256	1.423e-02	4.618e-02	3.25×	2.07e-12	1.36e-12	6.9e+09	8940×

Exact. Residual and Jacobian match an *independently written* strong-form full-grid path at machine precision, and eliminating the pressure agrees with the *real FOM pressure* along a path the solver never takes. Vector-valued, divergence-free, pressure-free machinery: verified.

Accurate — and beaten. The nonlinear ROM is a factor of 2.6–3.3 better than POD at the matched online dimension ($K = 8$, column 4). But the **direct reduced solve in the same G span** lands at round-off: up to twelve orders better in error, and $\sim 9\,000\times$ faster (last two columns).

Read it as a rehearsal. Stokes is linear with affine parameters, so by correction 2 the nonlinear head could only ever cost more for nothing — and it did, exactly as the design predicted. **This cell must not be written up as a positive result.** Its deliverable is the machinery Navier–Stokes needs.

Cost ladder 1/4 — Poisson 2D: flat, free, and nothing to fit

N	full grid		sampled (NNLS)		quadrature-free		setup s	
	pts	ms	pts	ms	pts	ms	NNLS	QF
128	15 876	3.98	256	3.14	0	2.93	28.6	0.9
256	64 516	4.96	256	2.75	0	2.68	37.3	0.7
512	260 100	11.11	256	3.09	0	2.87	25.4	0.8

Online solve ms per held-out source at $\tau = 10^{-3}$; all three paths through the identical solver, A100-80GB.

The $O(n)$ arm moves; the precomputed arm does not.
The full-grid path sums over every point of the mesh and its column climbs steadily with N . The quadrature-free path touches no grid point at all, and its column is flat across the whole ladder.

The offline columns matter as much. The sampled rule cannot be used until an NNLS fit has been run; the quadrature-free path needs one $M \times R$ matrix product. There is nothing to fit because there is nothing being approximated — and the sample-point columns say so directly.

Cost ladder 2/4 — 1D Burgers: flat, but so is everything else

One job, one A100-80GB, N interleaved (job 3033260) — latent-solve ms per 50-step trajectory

N	full-grid oracle	NNLS-32	tensor
128	21.4	16.5	17.1
256	26.2	19.9	20.2
512	21.7	14.8	15.7
1024	21.3	15.5	16.0
2048	20.4	15.9	14.4
4096	20.8	13.6	14.3
exponent p	-0.04	-0.07	-0.08

Beyond the launch floor — separate jobs: cross-job figures, not a fitted exponent

N	GPU	full-grid oracle	NNLS-32	tensor
16 384	A100-80GB	21.4	13.9	14.2
65 536	A100-80GB	32.9	13.5	13.8

No crossover in 1D, and the reason is not the method. Even the full-grid oracle looks flat below $\sim 10^4$ points: each solver iteration is a handful of tiny GPU kernels whose launch overhead dwarfs the arithmetic. Wall time counts *operations*, not grid points — the launch floor.

Where the grid finally shows. Only in the lower block does the oracle begin to climb while the tensor does not move. Those two rows come from *separate jobs*: they are cross-job figures, not a fitted exponent. The exponent in the upper block is fitted inside one job, where it means something.

Cost ladder 3/4 — 2D Burgers: a real crossover, on mixed hardware

<i>N</i>	GPU	job	full grid	exact-lin. + NNLS	tensor	tensor/full	tensor/NNLS
64	A100-40GB	3038943	32.6	28.8	27.5	0.845	0.955
256	A100-80GB	3038921	74.1	29.1	27.7	0.374	0.952
512	H200	3038922	97.3	20.5	20.0	0.205	0.975
1024	A100-80GB	3038923	670.0	29.4	27.1	0.040	0.924

Read the GPU column first: 3 different cards across 4 rungs. Each *N* is its own job on its own node, so the tensor/full and tensor/NNLS *ratios* (measured inside one job) are sound, while the raw ms *across* rows mix hardware and are not an exponent.

In 2D the grid is finally big enough to be felt. At the coarsest rung the full-grid residual is barely worse than the tensor — still launch-bound. By the finest rung it is more than an order of magnitude slower, and the tensor has not moved.

The confound is in the table, not in a footnote. The GPU column is second for a reason: these rungs did not share hardware. The tensor/full and tensor/NNLS ratios are measured *within* one job and are sound; the raw millisecond columns across rows mix cards, so no exponent may be fitted through them.

Cost ladder 4/4 — Stokes 2D: the ROM crosses the FOM, and still loses

N	FOM	full-grid ROM	EQ ROM	quad.-free ROM	direct G -span	QF / G -span
32	0.0015	0.1154	0.0533	0.0256	$4.8\ \mu\text{s}$	$5387\times$
64	0.0114	0.2829	0.0555	0.0342	$4.9\ \mu\text{s}$	$7038\times$
128	0.0904	3.0653	0.0629	0.0351	$5.2\ \mu\text{s}$	$6743\times$
256	0.4120	16.4207	0.0753	0.0517	$5.8\ \mu\text{s}$	$8940\times$

Seconds per query (median of 12 timed reps, gate S7); every timed path is NumPy on CPU, host NVIDIA GB10. The FOM column is the back-substitution of a pre-factored MAC system, i.e. the *cheapest honest* full-order query.

A crossover does exist here. The FOM column grows steeply with N — a bigger back-solve every query — and the full-grid ROM worse still. The quadrature-free ROM barely moves, and between the second and third rung it passes under the FOM and stays there.

And it still loses. The last column is that same quadrature-free ROM measured against the direct reduced solve in the identical span. Being flat in N is not the same thing as being the right algorithm for the problem.

Read the caption. These are CPU NumPy seconds, deliberately: the cell was about exactness and structure, not device tuning, and putting a launch-bound GPU path beside a microsecond direct solve would have measured the launcher instead of the method.

The pattern — and what is honestly not measured

PDE class	residual degree	per- N online-cost ladder	crossover vs the FOM
Poisson 2D	1 (linear)	measured, $N = 128/256/512$	not measured (soft: vs unpreconditioned CG)
Heat 2D/3D	1 (linear)	<i>not measured</i> direct reduced POD–Galerkin 13–38 $\times$ faster than the FOM (no N ladder behind it)	<i>not measured</i> ; only an aggregate:
Wave 2D	1 (linear, conservative)	<i>not measured</i>	<i>not measured</i> — cell retired unimplemented
Burgers 1D	2 (quadratic)	measured, $N = 128 \dots 65\,536$	no crossover: launch-bound below $\sim 10^4$ pts
Burgers 2D	2 (quadratic)	measured, $N = 64/256/512/1024$	$\approx N = 512$ (tensor flat, oracle $O(n)$)
Stokes 2D	1 (linear, vector, constrained)	measured, $N = 32/64/128/256$	$N = 128$ (QF ROM under the FOM back-solve)
Navier–Stokes	2 (quadratic)	not run yet	not run yet

The pattern, in one line. The precomputed residual is **flat in N** ; the full-grid residual is $O(n)$. That holds in every ladder above. Flatness is a property of the *method* — but where the two arms *cross* is a property of the problem and the machine, and it moves.

Where they cross. 1D: never — below the launch floor both arms are dominated by kernel launches. **2D Burgers: around $N = 512$** , where the $O(n)$ arm leaves that floor. **Stokes: at $N = 128$** , against a pre-factored full-order back-solve.

Two gaps, named not omitted. **Heat** has no per- N ladder; its one figure is an aggregate comparing a *direct reduced linear solve* with the FOM, not our residual with itself. **Waves** has no cost measurement at all — retired before implementation, because on a conservative PDE it is the decoder, not the residual, that fails.

What five adversarial reviews said (3 Claude, 2 Codex) — and where the method stops

Verdicts: “legitimate with corrections”
×5, none blocking

- No leakage: T from bank + modes only; oracle is the committed baseline path; truth regenerated from seed, test set seed-disjoint.
- One reviewer reproduced every printed digit on CPU.
- Corrections applied: “second order” → first order with a bound; “identical decisions” → 31/32; “beats NNLS by 1–9 %” retracted; arm order was sequential, not interleaved; cross-job exponents → ratios; f64 required.
- Framing: standard quadratic Galerkin/Oplnf operator; novelty is the MLP-head-over-learned-bank setting.

Where it stops

- **Sign-changing data:** the $|u|$ term is not tabulated. Routes: a polynomial FOM (centered / skew / Lax–Friedrichs), tabulate the central part and sample only the small $|u|$ term, or lift $v = |u|$.
- **Non-polynomial physics** (limiters, WENO, e^u): no finite table; sampling stays the right tool, or lift.
- **2D:** works as-is for the $R = 64$ bank. The $R = 512$ headline bank needs a 270 MB table; head-PCA compression failed (tried) — it stays sampled.
- **Statistics:** single seed, 8 trajectories; multi-seed with paired intervals is still open.

Linear vs nonlinear, side by side

	Linear (Poisson)	Quadratic, positive data (Burgers, tensor)	Non-polynomial / switched (sampled)
Residual in $h(z)$	$\Lambda B h - f_M$, linear	B -terms + $h^\top T h$, quadratic	linear terms via B ; the rest evaluated pointwise
Grid folded away?	entirely (B once)	entirely (B and T once)	only the linear terms
Online per iteration	head + 64×32 matvec	head + 32^3 contraction	head + decode at m nodes + stencil + matvec
Sample points	0	0	m ($32 / 16 / 128$)
Residual approximation	none (10^{-13})	none on $u \geq 0$; $O(\Delta x \delta)$ on under-shoots	quadrature error (NNLS + learned nodes)
After stage 1	nothing	nothing (build T once)	NNLS weights + stage-2 node positions
Accuracy limit	decoder floor	decoder floor (= oracle)	decoder floor + sampling gap
Cost vs grid size	flat	flat (1D solve $128 \rightarrow 65\,536$; 2D $64^2 \rightarrow 1024^2$ while the oracle grows $25\times$)	flat

One sentence

The decoder is linear in its last stage, so every *polynomial* piece of the physics can be pre-projected onto the test modes once — linear into a matrix, quadratic into a 3-index table; only a genuinely non-polynomial piece (a switch, a limiter, e^u) must be sampled, and there we learn where to sample.

Glossary (1/2)

- Latent z** The K numbers (8 or 16) that stand in for a whole field.
- Bank g , head h** The two halves of the decoder: shapes in space (g ; table G once evaluated) and mixing amounts from z (h).
- Test modes Φ , M** The M sine functions the residual is projected onto (weak form); eigenvectors of the discrete Laplacian with eigenvalues Λ .
- $B = \Phi^\top G$** The precomputed $M \times R$ table that turns any linear grid sum into a small matvec.
- Quadrature** Estimating a grid-wide sum from m sample points with weights. **Quadrature-free** = the sum is computed exactly without points.
- Full grid / oracle** The residual (Poisson) or the advection term (Burgers) summed over every grid point — the reference the sampled rules try to match; costs $O(N)$ per evaluation.
- Tensor T / Q** The precomputed $M \times R \times R$ table that gives $\Phi^\top(u u_x)$ as $h^\top T h$; Q its symmetrised form for the Jacobian. Built once offline; makes the quadratic term exact with zero sample points.
- Undershoot** Decoded field values slightly below zero where the truth is ≥ 0 ; the only place the tensor and the sign-upwind oracle differ.
- Launch floor** The regime where each GPU operation finishes faster than the next can be launched, so wall time counts operations, not data; every arm sits on it below $\sim 10^4$ points.

Glossary (2/2)

- NNLS** Non-negative least squares: fits the node weights so the sampled term reproduces the full-grid term on training states; convex, no gradients.
- Learned nodes** Node positions optimised by gradient (stage 2) with the decoder and teacher frozen; weights still by NNLS.
- Tight / starved / generous budget** $m = 32 (= M)$, 16, 128 nodes.
- Decoder floor** The error of the best possible latent for each truth snapshot; nothing downstream can beat it.
- Rollout error** Relative L_2 error against the full-order truth over the 50 implicit time steps, mean of 8 fresh test trajectories.
- Gate** An in-job numerical identity check (e.g. gate Q: quadrature-free residual = full-grid residual to machine precision at every solution; gate T0: tensor = oracle on all-positive states).
- LM attempt** One iteration of the Levenberg–Marquardt solver (residual + Jacobian + trial step); ~ 200 per 50-step trajectory.